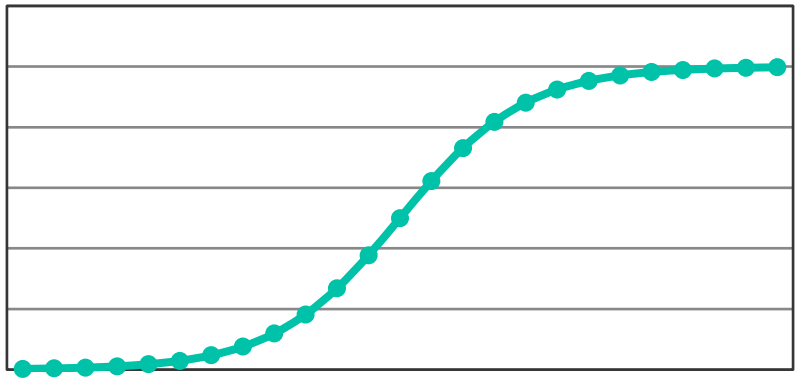
DEEP LEARNING FOUNDATIONS

Sigmoid, Softmax, ReLU & the functions that give neural networks their power



$$f(x) = 1 / (1 + e^{-x})$$

Prepared for: Neural Network Fundamentals

THE CORE IDEA

Why Activation Functions Matter

Without an activation function, a neural network — no matter how many layers — collapses into a single linear equation. Activation functions inject non-linearity, letting networks learn curves, boundaries, and hierarchies of features.

1

Non-linearity

Enables the network to approximate complex, curved decision boundaries instead of a flat line.

2

Gradient flow

Shapes how error signals propagate backward during training via backpropagation.

3

Output shaping

Squashes or transforms outputs into ranges suited to the task (probabilities, signed values, etc.).

4

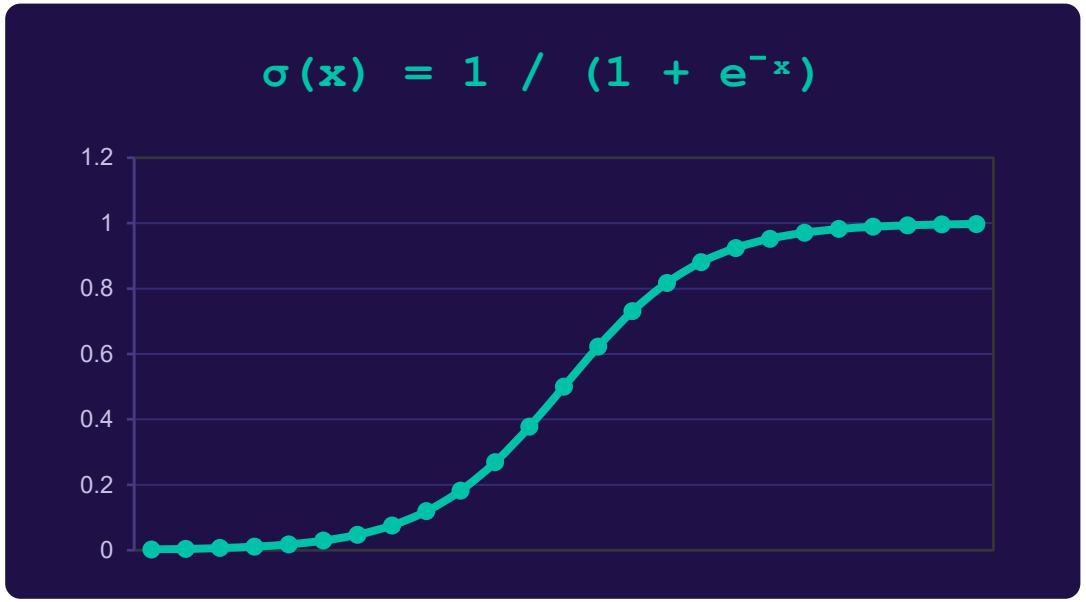
Representation power

Stacked non-linear layers let deep nets learn hierarchical features layer by layer.

Rule of thumb: hidden layers favor gradient-friendly, computationally cheap functions (ReLU family); output layers are chosen to match the task (sigmoid, softmax, linear).

FUNCTION 01

Sigmoid Activation Function



Significance

Squashes any real number into the range (0, 1), making outputs interpretable as probabilities. It was the original workhorse activation that first let networks learn non-linear patterns.

Where it's used

- Output layer of binary classifiers (e.g., spam vs. not spam)
- Gating mechanisms in LSTM / GRU cells (forget, input, output gates)
- Logistic regression models

Strengths

- Smooth, differentiable everywhere
- Clear probabilistic interpretation
- Bounded output stabilizes small networks

Limitations

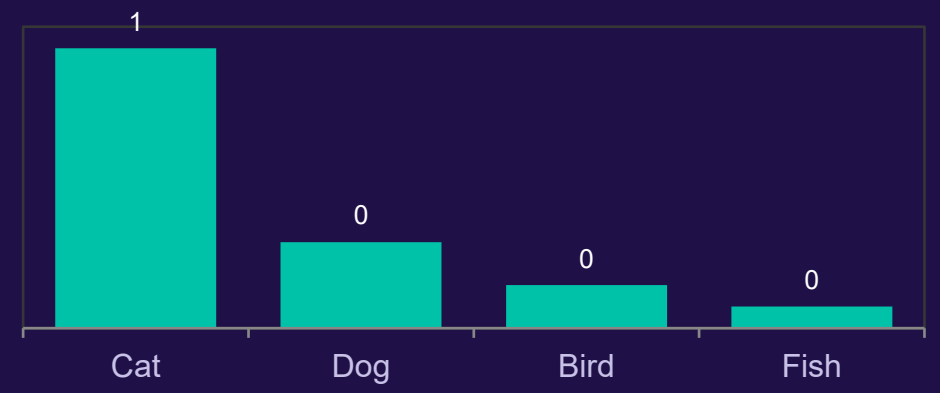
- Vanishing gradients for large |x|
- Output not zero-centered, slows convergence
- Costly exponential computation

FUNCTION 02

Softmax Activation Function

$$\text{softmax}(x_i) = e^{x_i} / \Sigma_j e^{x_j}$$

Example: classifying an image into 4 classes



Significance

Converts a raw score vector into a probability distribution: every value lands in (0, 1) and the whole vector sums to exactly 1. It generalizes sigmoid from two classes to many.

Where it's used

- Output layer of multi-class classifiers (image, text, speech)
- Attention weight computation in Transformers
- Paired with categorical cross-entropy loss for training

Strengths

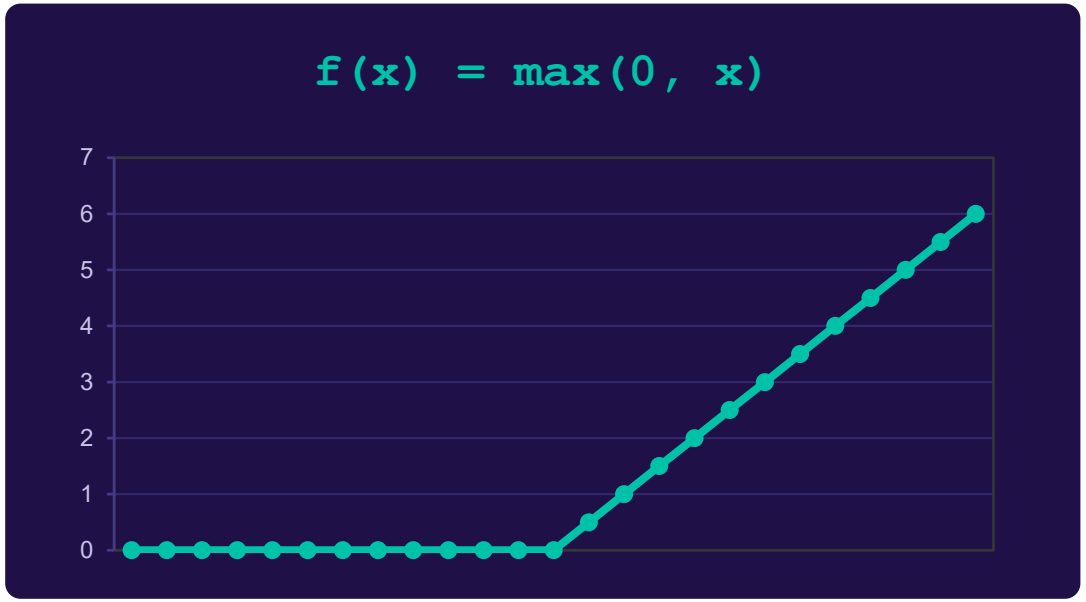
- Outputs a clean probability distribution
- Naturally handles any number of classes
- Differentiable, pairs well with cross-entropy

Limitations

- Sensitive to large logits / outliers
- Only suited to the final output layer
- Can overstate confidence on one class

FUNCTION 03

ReLU (Rectified Linear Unit)



Significance

Passes positive values through unchanged and zeroes out negatives. It's cheap to compute and keeps gradients strong for active neurons, which is why it became the default choice for deep networks.

Where it's used

- Default hidden-layer activation in CNNs (image recognition)
- Deep feed-forward and fully connected networks
- Most modern architectures as a fast baseline choice

Strengths

- Very cheap to compute (no exponentials)
- Mitigates vanishing gradients for x > 0
- Sparse activation improves efficiency

Limitations

- “Dying ReLU” — neurons can get stuck at 0
- Not zero-centered
- Unbounded output can grow very large

BEYOND THE BIG THREE

Other Important Activation Functions

Tanh

`tanh(x)`

Zero-centered version of sigmoid, range (-1, 1). Used in RNN/LSTM hidden states and when centered gradients help convergence.

Leaky ReLU

`max(0.1x, x)`

Allows a small negative slope so neurons never fully “die.” Used as a drop-in ReLU replacement in GANs and deep CNNs.

ELU

`x, or α(e^x−1)`

Smooth negative saturation pushes mean activations toward zero, speeding up learning in deep nets.

GELU

`x·Φ(x)`

Smooth, probabilistic gating used as the default in Transformer models like BERT and GPT.

Swish / SiLU

`x·σ(x)`

Self-gated, smooth and non-monotonic; found by search to outperform ReLU in EfficientNet-style networks.

Linear (Identity)

`f(x) = x`

No non-linearity; used in the output layer for regression tasks predicting continuous values.

BEYOND THE BIG THREE

Strengths & Limitations, In Detail

Function	Key Strength	Key Limitation
Tanh	Zero-centered (-1,1) output gives faster, more stable convergence than sigmoid.	Still saturates at extremes — same vanishing-gradient problem as sigmoid, plus costlier than ReLU.
Leaky ReLU	Small negative slope keeps gradients non-zero, so “dead” neurons can recover during training.	Adds a slope hyperparameter to tune; gains over plain ReLU are often only marginal.
ELU	Smooth negative saturation pushes mean activations toward zero, speeding up learning.	Exponential term for $x < 0$ makes it more expensive to compute than ReLU or Leaky ReLU.
GELU	Smooth, probabilistic gating improves optimization in very deep, wide Transformer stacks.	More costly than ReLU; its edge over simpler functions is small on small networks.
Swish / SiLU	Self-gated and non-monotonic; discovered via search to beat ReLU in EfficientNet-style nets.	More expensive per-activation than ReLU; gains are architecture-dependent, not guaranteed to transfer.
Linear	Preserves the full, unrestricted output range needed for continuous-value prediction.	Adds no non-linearity — stacking linear layers collapses to one layer, so never used in hidden layers.

Full walkthroughs — including where each function is used — are in the speaker notes of the previous slide.

SIDE BY SIDE

Comparative Overview

Function	Range	Typical Layer	Best Suited For	Key Drawback
Sigmoid	(0, 1)	Output	Binary classification, gates	Vanishing gradient
Softmax	(0, 1), sums to 1	Output	Multi-class classification	Sensitive to outliers
ReLU	[0, ∞)	Hidden	Default for CNNs / deep nets	Dying ReLU
Tanh	(-1, 1)	Hidden	RNN / LSTM states	Vanishing gradient
Leaky ReLU	(-∞, ∞)	Hidden	Fixing dying ReLU	Extra hyperparameter
GELU	≈(-0.17, ∞)	Hidden	Transformers (BERT, GPT)	More costly than ReLU
Swish / SiLU	≈(-0.28, ∞)	Hidden	Deep CNNs (EfficientNet)	More costly than ReLU
Linear	(-∞, ∞)	Output	Regression	No non-linearity

All functions are differentiable (or sub-differentiable) to support gradient-based backpropagation.

QUICK DECISION GUIDE

Hidden layers, general use?

Start with ReLU — fast and effective.

Hidden layers, dying neurons?

Switch to Leaky ReLU, ELU, or GELU.

RNN / LSTM hidden state?

Use Tanh for zero-centered outputs.

Output: binary classification?

Use Sigmoid for a probability score.

Output: multi-class classification?

Use Softmax for a probability distribution.

Output: regression (continuous value)?

Use Linear (identity) — no squashing.

Activation functions are the switchboard of a neural network — the right choice keeps gradients healthy and outputs meaningful.

MACHINE LEARNING FOUNDATIONS

The Bias–Variance Trade-off

Overfitting, Underfitting & Regularization Strategies

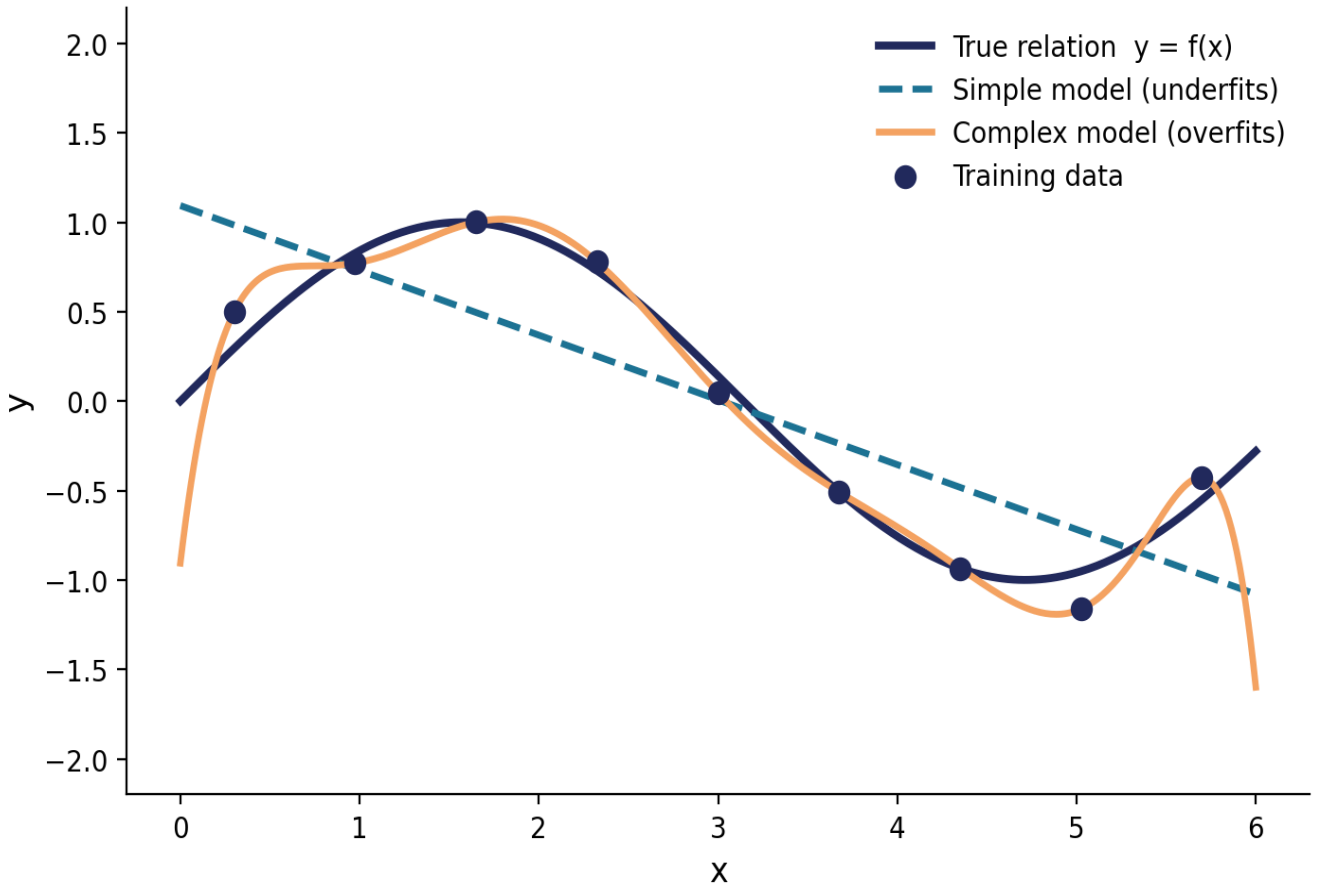
Covers · Bias-Variance · Overfitting & Underfitting · L2 Regularization · Dataset Augmentation · Early Stopping

AGENDA

What we'll cover

01	**The Core Problem**	Approximating an unknown true function f(x) with a model
02	**Bias & Variance**	Formal definitions, and how model complexity drives each
03	**Overfitting vs. Underfitting**	Reading training and validation error together
04	**Diagnosing Errors in Practice**	A practical playbook for train/validation error patterns
05	**Regularization Toolkit**	L2 penalty, dataset augmentation, and early stopping

The Core Problem: Approximating f(x)



True relation

$$y = f(x)$$

The unknown function we're really trying to learn.

Our approximation (model)

Simple (degree 1): $y = \hat{y}(x) = w_1 x + w_0$

Too rigid to bend toward the curve of f(x) → underfits.

Complex (degree 25):

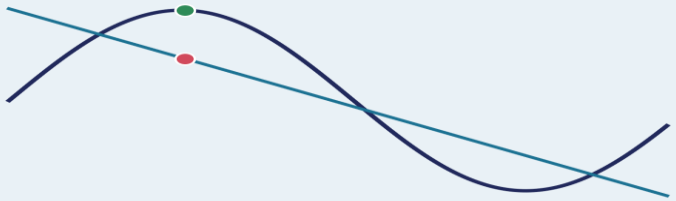
$$y = \hat{y}(x) = \Sigma\, w_i x^i + w_0 \quad (i = 1 \ldots 25)$$

Flexible enough to chase noise in the training points → overfits.

Underfitting vs. Overfitting

HIGH BIAS

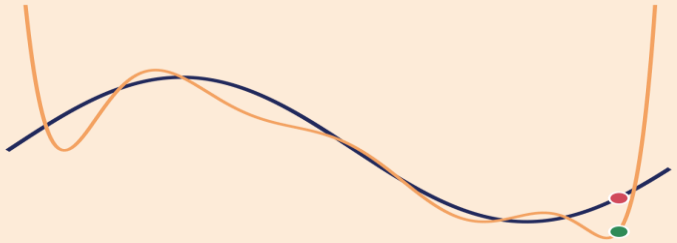
Underfitting



- Model is too simple to capture the underlying pattern.
- High error on both training and test data.
- Example: fitting a straight line through a sine wave.

HIGH VARIANCE

Overfitting



- Model is too flexible and memorizes noise in training data.
- Low training error, but high error on unseen (test) data.
- Example: a high-degree polynomial wiggling through every point.

Reading the dots

True value f(x)

Model prediction

A wide gap between the dots means the model's prediction misses the true relationship — whether from being too rigid (bias) or too erratic (variance).

FORMAL DEFINITIONS

Bias & Variance, Defined

BIAS

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

VARIANCE

$$\text{Variance}(\hat{f}(x)) = E\left[\left(\hat{f}(x) - E[\hat{f}(x)]\right)^2\right]$$

f(x) is the true function; the hatted term is our model's prediction of it.

Simple Model

High bias, low variance

Underfits: too rigid, misses real structure in the data.

Complex Model

Low bias, high variance

Overfits: fits training noise, unstable across datasets.

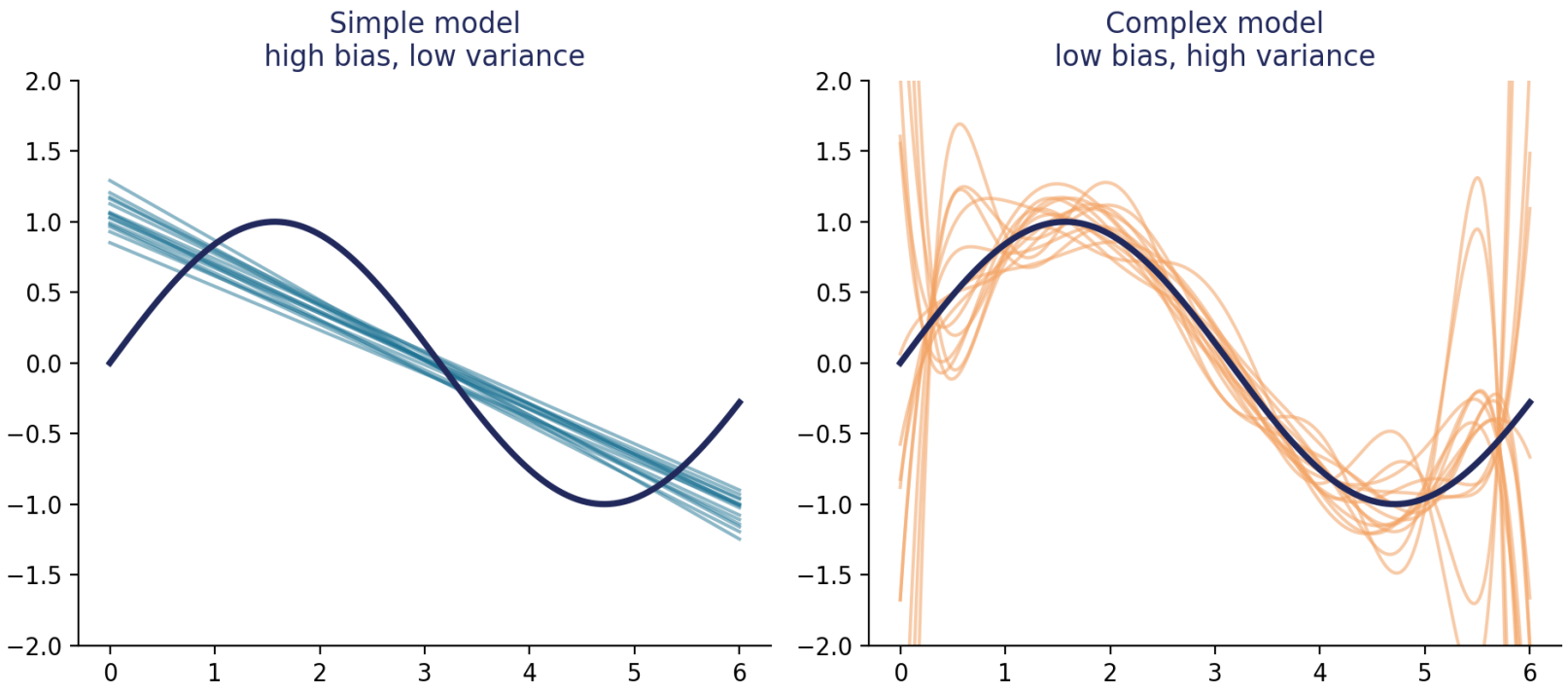
Ideal Model

Low bias, low variance

The trade-off sweet spot every ML workflow is chasing.

WHAT HAPPENS IF YOU TRAIN ON DIFFERENT SETS OF TRAINING POINTS?

Visualizing Variance Across Datasets



Simple model

Lines barely move between runs — consistent, but consistently wrong (high bias, low variance).

Complex model

Curves swing wildly between runs — each fits its own sample closely, but they disagree with each other (low bias, high variance).

Each curve is the same model re-trained on a different random sample from the same true function f(x) = sin(x).

HOW DO ERROR CURVES MOVE WITH MODEL COMPLEXITY?

The Bias–Variance Trade-off Curve



Sweet Spot

Left of it → **high bias** dominates (underfitting)

Right of it → **high variance** dominates (overfitting)

At it → **perfect trade-off, ideal model complexity**

Goal: pick model complexity (or apply regularization) to land as close to the sweet spot as possible.

HOW DO WE DEAL WITH THIS IN PRACTICE IN DNNS?

Diagnosing Model Errors

Divide data into train / validation / test. Start with a baseline network, use the right activation (tanh/ReLU/leaky ReLU), initialization (Xavier/He), and optimizer (Adam) — then read the errors.

Training Error	Valid Error	Cause	Solution
High	High	**High bias**	Increase model complexity · Train for more epochs
Low	High	**High variance**	Add more data (dataset augmentation) · Regularization · Early stopping
Low	Low	**Perfect trade-off**	You are done!

HOW DO WE DEAL WITH THIS IN PRACTICE IN DNNS?

The Practical Workflow

1	**Split your data**	Divide data into train, validation/development, and test splits before doing anything else.
2	**Start with a baseline**	Pick some initial network configuration — e.g., 2 hidden layers, 50 neurons each.
3	**Use sound defaults**	Right activation function (tanh, ReLU, leaky ReLU), right initialization (He, Xavier), and right optimizer (e.g., Adam).
4	**Monitor, don't peek**	Track training and validation error as you go — but do not touch the test data until the very end.

The Learning Algorithm

Initialise w, b

Iterate over data:

compute $\hat{y}$

compute $\mathcal{L}(w, b)$

$w_{111} = w_{111} - \eta \cdot \Delta w_{111}$

$w_{112} = w_{112} - \eta \cdot \Delta w_{112}$

....

$w_{313} = w_{313} - \eta \cdot \Delta w_{313}$

till satisfied

$$min \ \mathcal{L}(\theta) = \mathcal{L}train(\theta) + \Omega(\theta)$$

$$\mathcal{L}train(\theta) = \Sigma \ (y_i - f(x_i))^2 \quad i = 1...N$$

$$\Omega(\theta) = W^2_{111} + W^2_{112} + \cdots + W^2Lnk$$

$$\Delta W_{ijk} = \partial \mathcal{L}(\theta)/\partial W_{ijk}$$

$$= \partial \mathcal{L}train(\theta)/\partial W_{ijk} + \partial \Omega(\theta)/\partial W_{ijk}$$

The regularization penalty Ω(ϑ) is simply added into the loss — so every gradient step now also pulls weights gently toward zero.

Regularization: The L2 Penalty

Add a penalty term Ω(θ) to the loss so the optimizer is discouraged from pushing weights to extreme values that only help fit training noise.

$$\mathcal{L}train(\theta) = \Sigma\,(y_i - f(x_i))^2 \qquad i = 1 \ldots N$$

$$min\ \ \mathcal{L}(\theta) = \mathcal{L}train(\theta) + \Omega(\theta)$$

$$\theta = [W_{111}, W_{112} + \cdots + WLnk]$$

$$\Omega(\theta) = \|\theta\|^2_2 = W^2_{111} + W^2_{112} + \cdots + W^2Lnk$$

Why it works

Penalizing large weights keeps the learned function smoother and less sensitive to individual data points — trading a little training error for much better generalization.

Gradient update changes

$$\Delta W_{ijk} = \partial\mathcal{L}train(\theta)/\partial W_{ijk} \ + \ \partial\Omega(\theta)/\partial W_{ijk}$$

Every weight update now also gets nudged toward zero, on top of the usual gradient step.

Regularization: Dataset Augmentation

1

Too little data, too many parameters

A complex model can easily drive training error to zero when the dataset is small — it simply memorizes every point.

2

More data raises the bar

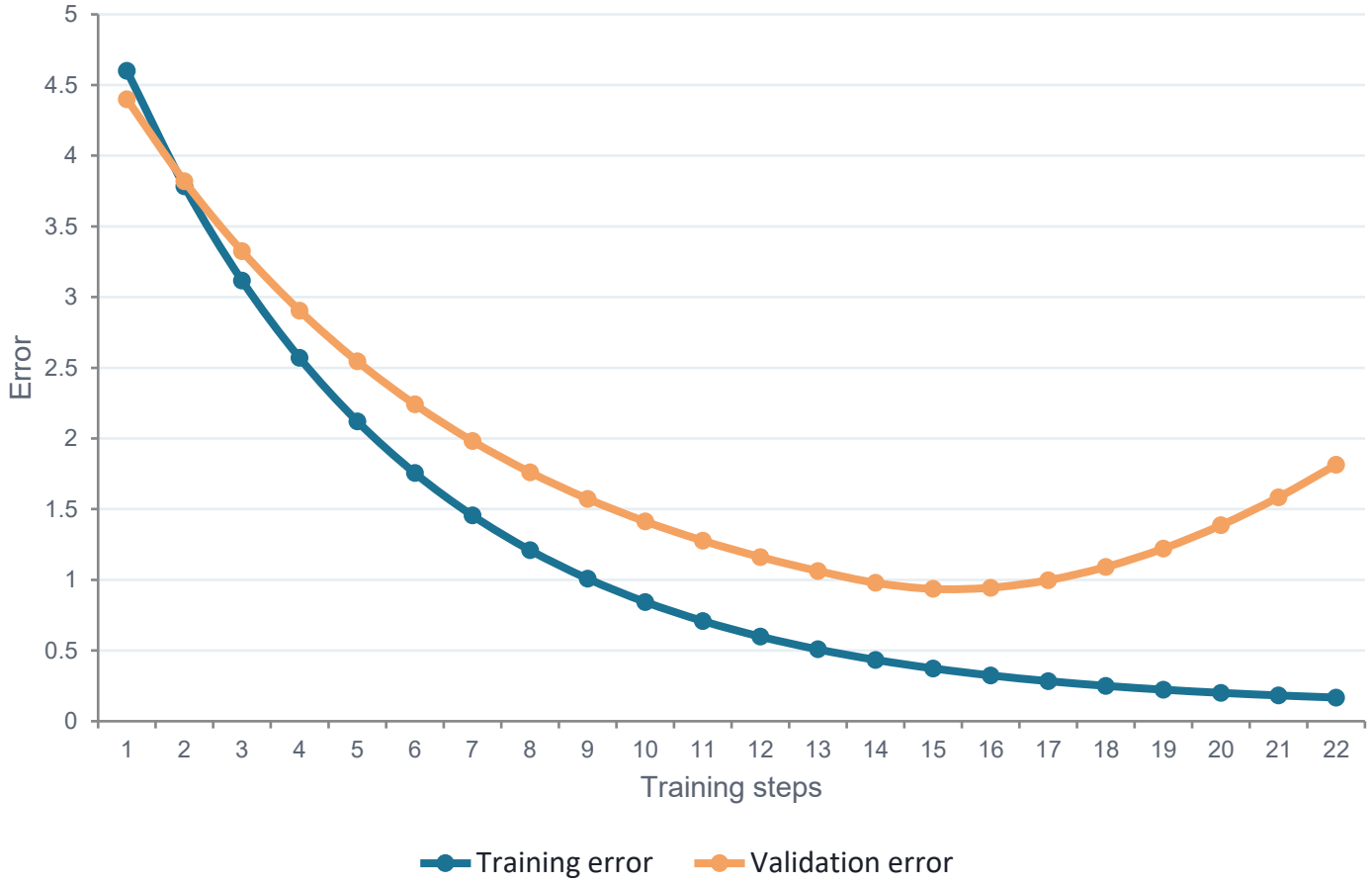
Augmenting with more (realistic, varied) data makes it harder for the model to drive training error all the way to zero — it can't memorize what it hasn't seen distinctly.

3

Augmented data resembles held-out data

By augmenting, we may end up generating examples similar to validation/test data — effectively giving the model a preview of the distribution it will be judged on.

Regularization: Early Stopping



k – p: return this model

Validation error is at its minimum here. This is the checkpoint we keep — it generalizes best.

k: stop training

Beyond this point training error keeps falling, but validation error climbs — a clear sign of overfitting. Halt here.

IS THE CONCEPT OF TRAIN/VALID ERROR ALSO RELATED TO HYPERPARAMETER TUNING?

Hyperparameter Tuning (a small detour)

Algorithms

- Vanilla / Momentum / Nesterov GD
- AdaGrad
- RMSProp
- Adam

Network Architectures

- Number of layers
- Number of neurons

Activation Functions

- tanh (RNNs)
- relu (CNNs, DNNs)
- leaky relu (CNNs)

Strategies

- Batch
- Mini-Batch (32, 64, 128)
- Stochastic
- Learning rate schedule

Initialization Methods

- Xavier
- He

Regularization

- L2
- Early stopping
- Dataset augmentation
- Drop-out
- Batch Normalization

Everything on this slide is a knob you tune by watching the same train/validation error curves — regularization is just one branch of a much larger tree.

KEY TAKEAWAYS

Bringing it all together

Every model trades bias against variance

Simple models underfit (high bias); complex models overfit (high variance). The goal is the sweet spot.

Read train vs. validation error to diagnose

High-high = high bias → grow the model. Low-high = high variance → regularize.

Regularization is a toolkit, not one trick

L2 penalties, dataset augmentation, and early stopping each fight variance in a different way.

Never tune against the test set

Use train/validation to make every decision; the test set is the final, untouched check.

DEEP LEARNING · UNIT NOTES

Loss Functions

MSE · Binary Cross-Entropy · Categorical Cross-Entropy

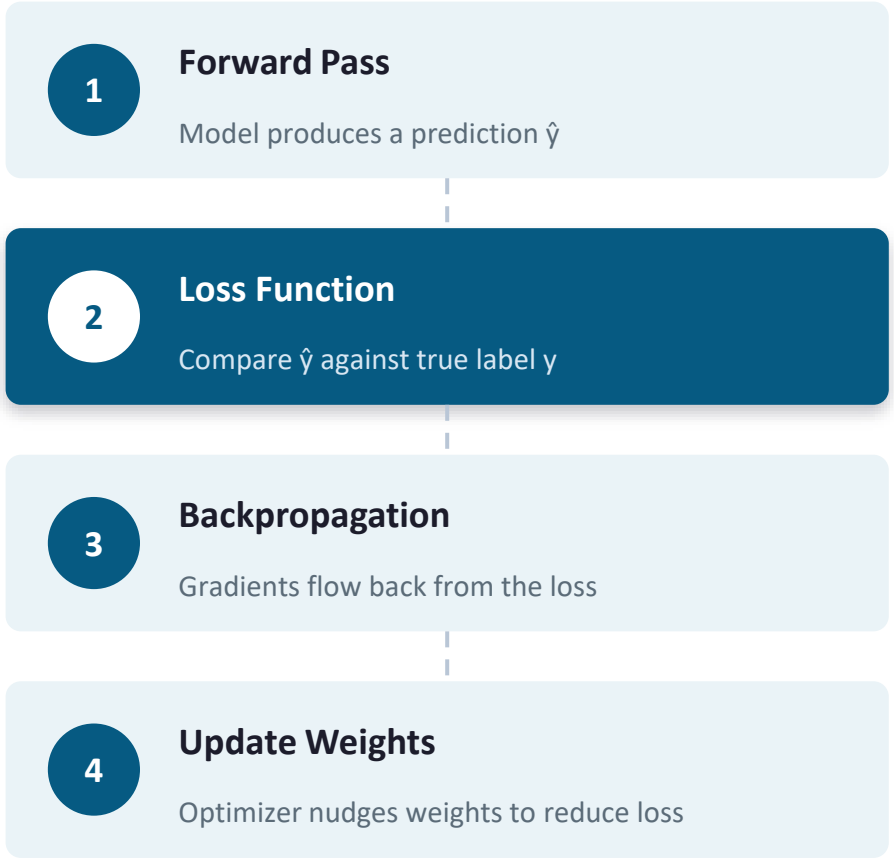
How a model measures its own mistakes — and how to pick the right ruler for the job.

THE CORE IDEA

What Is a Loss Function?

A loss function is a single number that scores how far a model's prediction is from the true answer.

- Small loss → prediction close to the truth
- Large loss → prediction far from the truth
- Training = adjusting weights to push this number down



LOSS FUNCTION 01

Mean Squared Error (MSE)

REGRESSION

$$L_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Averages the squared gap between predicted and true numeric values — larger errors are punished much harder than small ones.

SIGNIFICANCE

- Squaring removes sign, so over- and under-shoots both count as error
- Penalizes large errors heavily — pushes the model to avoid big misses
- Smooth, differentiable curve — gradients are easy and stable to compute

GOOD TO KNOW

- Used for continuous targets: price, temperature, age, coordinates
- Sensitive to outliers — one bad point can dominate the loss
- Paired with a linear (no activation) output layer

LOSS FUNCTION 02

Binary Cross-Entropy (BCE)

BINARY CLASSIFICATION

$$L_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Measures how well a predicted probability matches a 0/1 label — confident, wrong predictions are penalized steeply.

SIGNIFICANCE

- Rewards confident, correct probabilities; punishes confident, wrong ones
- Derived from maximum likelihood — a principled fit for probabilities
- Gradient stays strong even when predictions are very wrong

GOOD TO KNOW

- Used for two-class problems: spam vs not spam, tumor vs healthy
- Requires a sigmoid output so ŷ lies strictly between 0 and 1
- Also called log loss in classical machine learning

LOSS FUNCTION 03

Categorical Cross-Entropy (CCE)

MULTI-CLASS CLASSIFICATION

$$L_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

The multi-class extension of cross-entropy — only the log-probability assigned to the true class contributes to the loss.

SIGNIFICANCE

- Generalizes BCE from 2 classes to C mutually exclusive classes
- Directly optimizes the probability given to the correct class
- Works naturally with one-hot encoded label vectors

GOOD TO KNOW

- Used for single-label, multi-class tasks: digit or object recognition
- Requires a softmax output layer so class probabilities sum to 1
- Use sparse categorical cross-entropy if labels are integers, not one-hot

SIDE BY SIDE

Comparing the Three Losses

	MSE	Binary Cross-Entropy	Categorical Cross-Entropy
Task type	Regression	Binary classification	Multi-class classification
Output activation	Linear (none)	Sigmoid	Softmax
Target format	Real-valued number	0 or 1	One-hot vector
Penalizes	Squared distance	Wrong-confidence, log scale	Wrong-confidence, log scale
Typical use	Price / demand forecast	Spam / fraud detection	Image / digit classification

DECISION GUIDE

How to Choose Your Loss Function

Start from the shape of your output — the right loss almost always follows from it.

1

Is the output a continuous number?

e.g. House price, temperature, stock value

USE

Mean Squared Error

Linear output

2

Exactly two possible classes?

e.g. Fraud / not fraud, disease present / absent

USE

Binary Cross-Entropy

Sigmoid output

3

One of many mutually exclusive classes?

e.g. Handwritten digit, animal species, news topic

USE

Categorical Cross-Entropy

Softmax output

SUMMARY

Key Takeaways

1

The loss function defines the goal

It's the single number backpropagation tries to minimize — choosing it wrong means optimizing for the wrong thing.

2

Match the loss to the output layer

Linear → MSE, Sigmoid → BCE, Softmax → CCE. The activation and loss are chosen as a pair.

3

Cross-entropy for classification, MSE for regression

This single rule of thumb resolves the vast majority of real-world model choices.